mb-free___faults-wait

An AgentMPI run analysis

Abstract

This is the `WAIT` point of the four-policy barrier-failure sweep in `bench_faults` (`experiments/microbench.py`): eight ranks, rank 3 returns immediately to simulate a crashed agent session, and the seven survivors enter a barrier with a 15 s deadline. `WAIT` is the sweep's intended control — MPI semantics, block indefinitely — and `bench_faults`'s own docstring says it "is included precisely because it is MPI's only option and it never completes — which is the point." The run completed in 15.260 s. It did not block indefinitely, and the trace shows that it could not have: `BarrierPolicy.WAIT` has no distinct implementation anywhere in AgentMPI. `algorithms.barrier` switches to the arrival-counting `central` algorithm only for `PROCEED`, `SHRINK` and `REVOKE`, so `WAIT` runs the `dissemination` barrier, which never reads the policy at all and simply threads the caller's 15 s timeout into every send and receive; and on the one path where the policy *is* read, `WAIT` shares a branch verbatim with `RAISE`. Grepping `src/agentmpi` finds `BarrierPolicy.WAIT` exactly once, in that shared condition. The consequence is measurable rather than theoretical: this run and `mb-free__faults-raise` have identical event-kind counts (45 events; 17 `msg.send`, 10 `msg.recv`, 8 `rank.init`, 8 `rank.finalize`), identical per-rank send and receive profiles, a character-identical 17-edge communication matrix, an identical error string, and wall clocks 11 ms apart. The single most important thing to take away is that the sweep has three arms and not four, and that its control arm is a duplicate of its default arm. Any comparison in the paper's fault table between the `wait` and `raise` rows is reporting scheduler noise.

1 What this run is

property	value
run	`mb-free__faults-wait`
experiment	-
campaign label	-
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	45
wall time	15.26 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	17	17 eager, 0 rendezvous
tokens sent	17	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

Nothing anomalous was detected mechanically.

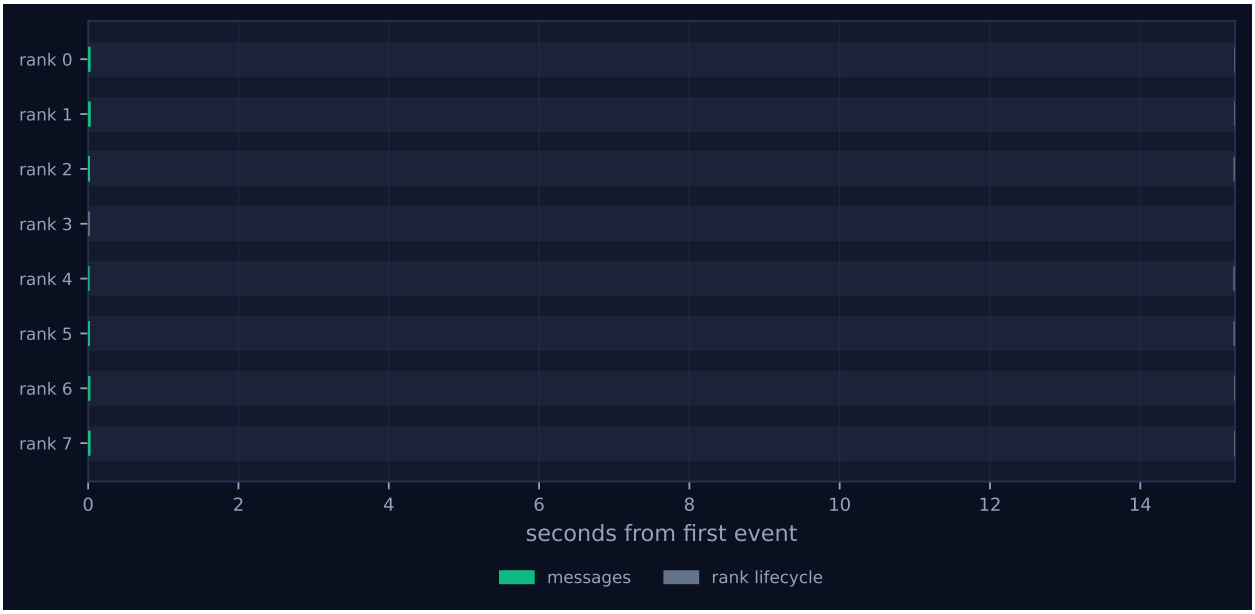

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

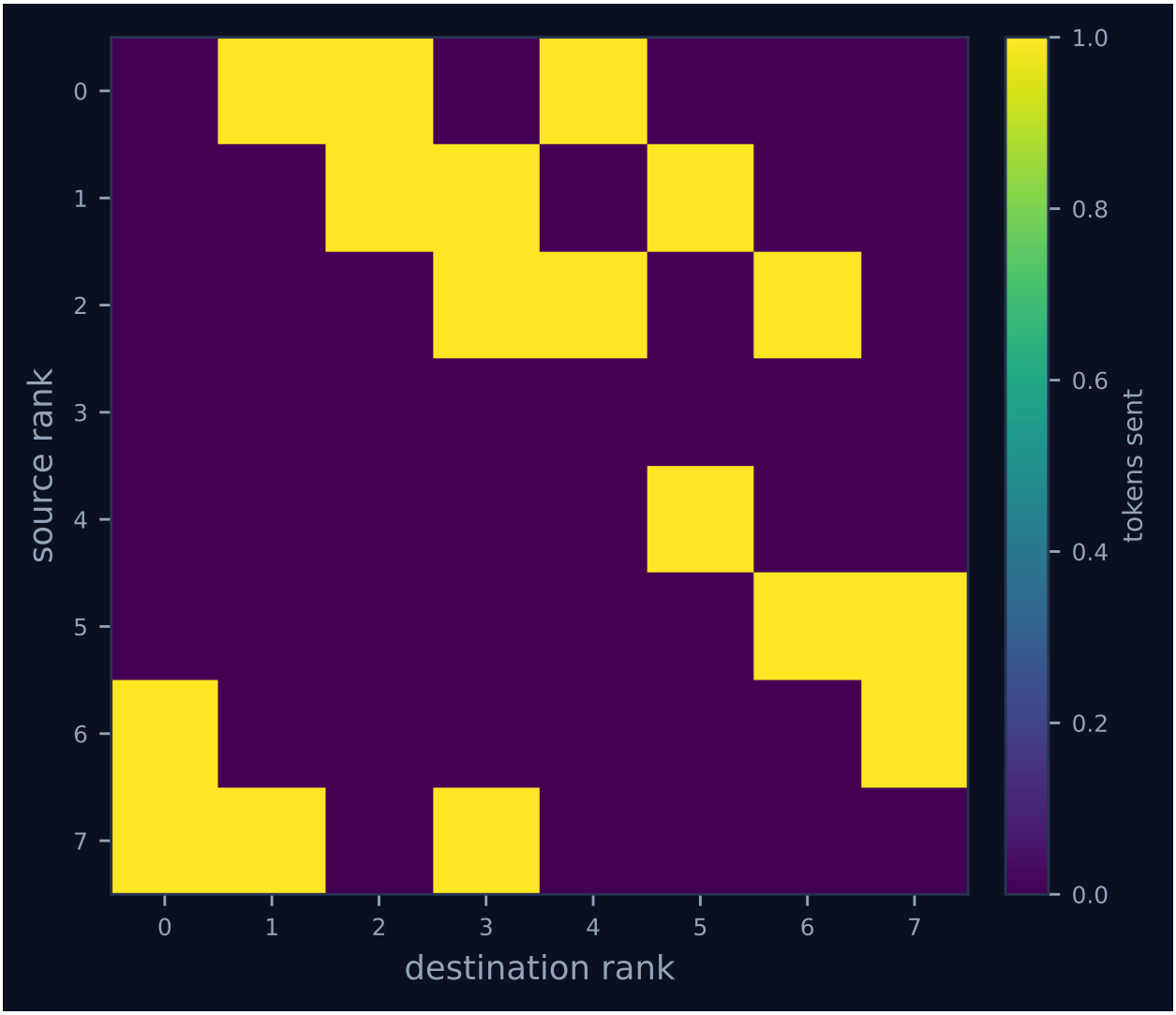

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	3	2	3	0.0	finalized
1	0.00	0.0	0	0	3	2	3	0.0	finalized
2	0.00	0.0	0	0	3	2	3	0.0	finalized
3	0.00	0.0	0	0	0	0	0	0.0	finalized
4	0.00	0.0	0	0	1	0	1	0.0	finalized
5	0.00	0.0	0	0	2	1	2	0.0	finalized
6	0.00	0.0	0	0	2	1	2	0.0	finalized
7	0.00	0.0	0	0	3	2	3	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

This run invoked no collectives.

6 Reading the timeline

Figure 1 has two vertical columns and nothing between them: a green cluster hard against $t = 0$ and a grey column at $t \approx 15.25$ s. The green marks are the barrier's 17 messages, all sent inside the first 20 ms; the grey marks are `rank.finalize`. There are no bars, because there is no work. `metrics.json` records `executors: {none: 8}`, zero agent calls, zero tokens and $0.0000, so the headline table's *achieved parallelism* 0.00× and *idle fraction 100.0%* are properties of the benchmark's design and not findings about the protocol: `bench_faults`'s `rank_main` calls `comm.barrier` and nothing else, and a rank with no task cannot have a busy fraction. Read both rows as "not applicable". The 15.260 s wall clock is likewise a constant — 15.0 s of `-detect-timeout` from `results/microbench/free-faults.json` plus 260 ms of process start-up and SQLite writes.

Rank 3's lane holds one tick at the far left: `rank.init` and `rank.finalize` both at $t = 0.012$ s. That is the injected fault in its entirety, and it is why the axis stops at 15 s rather than running to the job timeout.

The structure worth reading is compressed into the first 20 ms and the per-rank table in `generated.tex` resolves it better than the figure does. The *sent* and *recv* columns are not uniform:

rank	sent	recv	where it stopped
3	0	0	the victim; never entered the barrier
4	1	0	blocked in round 0, waiting on rank 3
5	2	1	blocked in round 1, waiting on rank 3
6	2	1	blocked in round 1, waiting on rank 4
0, 1, 2, 7	3	2	blocked in round 2, waiting on ranks 4, 5, 6 and 3

A dissemination barrier at $p = 8$ runs $\lceil \log_2 8 \rceil = 3$ rounds; in round k rank r sends to $(r + 2^k) \bmod 8$ and receives from $(r - 2^k) \bmod 8$. Rank 4's round-0 source is the dead rank, so it sends once and

stops; rank 5's round-1 source is rank 3 and rank 6's is the already-stalled rank 4, so both manage two sends; the other four complete two full rounds and stall in round 2. Summing gives $7+6+4 = 17$ sends against the 24 a complete barrier needs, and $6 + 4 = 10$ receives. The single injected failure reaches every survivor in exactly the three rounds the algorithm takes.

Figure 2 shows the same fact spatially. Row 3 is empty across its width; row lengths elsewhere encode blocking depth (three cells for ranks 0, 1, 2 and 7, two for ranks 5 and 6, one for rank 4). Column 3 carries three cells, from ranks 2, 1 and 7 — three messages delivered one per round into a dead rank's mailbox. Seven of the seventeen messages were never received: three by rank 3 and four by ranks 4, 5 and 6, which were alive but had already stopped reading. Nothing in the run notices either category, and neither category appears in any generated artefact; I recovered the count by differencing 17 sends against 10 receives and then reconstructing the schedule from the internal tags.

The viewer's stats row reads MESSAGES 17 and FAILURES 0, its legend offers only *messages 27* and *rank lifecycle 16*, and there is no COLLECTIVES panel at all, because no `coll.*` event was emitted. Its rank table lists all eight ranks as `finalized`, `alive: no`, `suspected: -`, with rank 3 indistinguishable from the seven that outlived it by fifteen seconds. Placed beside the `mb-free__faults-raise` dashboard the two are not merely similar; without the title bar there is nothing in either screenshot that distinguishes them.

7 Interpretation

7.1 `WAIT` is a synonym for `RAISE`, and the trace shows it twice over

The control-flow argument is short. `algorithms.barrier` opens with

```
if policy in (BarrierPolicy.PROCEED, BarrierPolicy.SHRINK, BarrierPolicy.REVOKE):
                                algorithm = "central"
```

so `WAIT` keeps the signature's `"dissemination"` default that `bench_faults` did not override. The dissemination branch is a loop of `comm._csend` and `comm._crecv` calls with `timeout=timeout` threaded through, and it contains no reference to `policy` whatsoever. The only code that reads the policy is `_central_barrier`, which this run never enters — and *that* code disposes of `WAIT` with

```
if policy is BarrierPolicy.RAISE or policy is BarrierPolicy.WAIT: raise
                                AmpiTimeout(...)
```

which is the only occurrence of `BarrierPolicy.WAIT` in `src/agentmpi`. There is therefore no code path in AgentMPI on which `WAIT` and `RAISE` differ, on either barrier algorithm.

The empirical confirmation is unusually complete, because the two runs differ in exactly one enum value and nothing else. Comparing the two `metrics.json` files programmatically: the `kind_counts` dictionaries are equal (`{job.create: 1, job.finish: 1, msg.recv: 10, msg.send: 17, rank.finalize: 8, rank.init: 8}`), the `role_counts` are equal (`lifecycle 18, message 27`), the per-rank `(sent, recv, tokens_sent)` triples are equal for all eight ranks, and the `comm` matrices are equal edge for edge — the same 17 ordered pairs at one token each. `results/microbench/free-faults.js` records the same error string for both rows, `ERR_TIMEOUT: receive timed out [ctx=0 rank=0 source=4 tag='_ampi:barrier:0:1:2' waited_s=15.0]`, and the same `detected_correctly:`

`false`. What separates the two runs is 11 ms of wall clock (15.260 versus 15.271 s), 1 ms of detection median (15.250 versus 15.249 s), and the interleaving of the 17 sends, which differ in which rank got the fabric's write lock first. The fabrics agree too: both end with an empty `failures` table, an empty `collectives` table, `comms.generation = 0`, `revoked = 0`, and all eight `comm_members` rows `active`.

7.2 What "block indefinitely" would actually have required

The enum's docstring is not wrong about what `WAIT` is *for* — "block indefinitely (MPI semantics; useful only for debugging)" is a coherent and useful semantic. It is wrong about where that semantic lives. Blocking indefinitely is what AgentMPI does when `timeout` is `None`, not when `policy` is `WAIT`: `_central_barrier` sets `deadline = None if timeout is None else time.time() + timeout` and its loop then has no exit but success, and on the dissemination path `_crecv(timeout=None)` blocks in `Communicator.recv` indefinitely. Both are reachable. Neither is reached by naming the policy.

So the harness could have built the control it wanted, but not by the parameter it used. `bench_faults` passes `timeout=cfg.detect_timeout` uniformly to all four policies, which is the right thing to do for the other three — comparing detection behaviour across policies requires holding the deadline fixed — and is precisely what prevents the fourth from being a control. A genuine `WAIT` arm would need `timeout=None`, and would then have been terminated not by the barrier but by `ampi.launch`'s job timeout, which `bench_faults` sets to `cfg.detect_timeout + 120` = 135 s. That is an inference from the harness source rather than something this trace shows, but it is the shape the missing arm would have had: a 135 s run ending in a job-level abort rather than a 15 s run ending in seven receive timeouts.

I would call the discrepancy a real defect, though the mildest of the three this sweep turns up. It is not a wrong computation; it is a parameter that is silently ignored, a docstring that describes behaviour the code does not implement, and a benchmark whose published claim about this row ("it never completes") is contradicted by the row's own `detect_p50_s` of 15.25 s. The cost is not to any running program — a harness asking for `WAIT` gets `RAISE`, which is at least bounded — but to the evidence. A four-row fault table with a control that is secretly a duplicate overstates what the table establishes.

7.3 What the run does establish on its own terms

Set the naming aside and this run is still informative, as the shared measurement of MPI's own semantics under a single fail-stop rank.

One dead rank at $p = 8$ costs all seven survivors, and the propagation is logarithmic. Every survivor blocked, none partially recovered, and the failure reached the last of them in the third and final round. This is the pathology `algorithms.barrier`'s docstring names — "the probability that all p members arrive within any fixed window falls off with p" — observed at the smallest population where the recursion has room to develop. It does not get better with scale: the round count is $\lceil \log_2 p \rceil$, so a larger population delays the collapse by a round or two rather than containing it.

The wait is bounded and every deadline is honoured. All seven survivors finalised between 15.247 and 15.259 s. Nothing hung, and there is no interval in this trace whose length is not a configured constant. Against real MPI, where a barrier with a dead peer hangs forever, that is the substantive difference, and it is worth noting that this run demonstrates it *despite* being the arm nominally

configured to reproduce the hang.

But the diagnosis names the wrong rank, and there is no failure record at all. The error reports `source=4`; rank 0's round-2 source is $(0 - 4) \bmod 8 = 4$, and rank 4 was alive throughout, finalising normally at $t = 15.247$ s. It must name a live rank: a dissemination barrier gives each participant visibility of exactly three peers, and rank 3 is not one of rank 0's. The benchmark's `detected_correctly` field encodes the consequence directly — it compares each survivor's detected list against the victim list `[3]`, and the fallback `ampi.get_failed(comm)` returns an empty tuple against an empty `failures` table, so every survivor reports nothing and the row scores `false`.

That empty table also closes off recovery. `ft.shrink` begins with `_agree_survivors`, which builds the survivor set from `get_failed` unioned with whatever `ft.health`'s lease detector suspects, and `ft.agree` computes its `expected` set the same way. With nothing declared, both would reconstruct a group still containing rank 3 and block on it again. A harness that catches this run's `AmpiTimeout` and tries to recover has nothing to recover with.

7.4 The flagged findings

The generated list reads: *"Nothing anomalous was detected mechanically."* True as defined, and badly wrong as read, and the reason is the same emptiness.

`Analysis.degraded` is `bool(failed_ranks) or ok is False or bool(incomplete_collectives)`. There are no incomplete collectives because `n_collectives` is 0 — the collective section of `generated.tex` says "This run invoked no collectives" about a run that invoked exactly one, since only `_central_barrier` calls `_record_collective` and only `_Tracker.finish` emits `coll.barrier`, and neither ran. `failed_ranks` is copied from `job.finish`, which lists ranks whose `rank_main` raised, and `bench_faults` catches `ampi.AmpiError` inside `rank_main`, so the job reported `ok: true` with an empty list. Even `coordination_is_underreported` is `false` here, which is correct by its own definition — no completed collective is having its time undercounted — and misleading in effect, because $7 \times 15.2 \approx 106$ rank-seconds of blocking inside a barrier are absent from every coordination figure and the flag that would have warned a reader stays down. The `PROCEED` and `SHRINK` siblings, which did record their barrier, both raise the flag and both report `degraded: true`.

The uncomfortable comparison this contributes to the sweep is that the two policies which handled the failure well are flagged as degraded and the two that handled it worst pass every mechanical check. `analysis.py` is not at fault — it can only score what the log holds — but the fix belongs upstream, in having the dissemination barrier record its invocation and its timeout the way the central one does.

7.5 Against its siblings, and what a harness author should do

The four runs are one program with one enum changed, so every difference is attributable to the policy:

policy	algorithm	events	wall (s)	msgs	`ft.declare_failed`	absentee named
`wait`	dissemination	45	15.26	17	0	no
`raise`	dissemination	45	15.27	17	0	no
`proceed`	central	39	15.22	0	7	yes
`shrink`	central	60	75.43	0	14	yes, then seven false ones

The first two rows are the same run twice. `PROCEED` detects correctly in 15.195 s, writes one durable failure row — seven `ft.declare_failed` events deduplicated by `ON CONFLICT` into a single `failures` entry for rank 3 — leaves the communicator intact at generation 0, and hands the harness the absentee list through a normal return. `SHRINK` detects identically and then attempts a repair that breaks: seven concurrent `UPDATE comms SET generation = generation + 1` statements in `ft.shrink_in_place` drive the generation from 0 to 7, the seven survivors cache six different values, and because `Communicator._record_collective` keys a collective on `(ctx, generation, epoch, op)`, the `ft.agree` that follows splits across six separate collectives, blocks for its full 60 s, and ends with seven false failure declarations naming healthy ranks and five ranks returning `true` over `voters: []`.

The recommendation is `PROCEED` wherever a phase's input can legitimately be partial and `REVOKE` plus an explicit out-of-place `ft.shrink` wherever it cannot; `RAISE`, which is the default, should be treated as unsuitable for any barrier a harness intends to recover from; and `WAIT` should not be used at all, because asking for it gets you `RAISE` without saying so. The evidence from this run for the last clause is as direct as evidence gets: the two traces are the same trace. For the first clause, note that `PROCEED` costs the same fifteen seconds this run costs and buys strictly more — an absentee list, a durable failure record, and zero messages of traffic where this run sent seventeen. `REVOKE` is untested in the sweep, which is a genuine gap, but it is the only escalating policy that does not route through `shrink_in_place`: it writes an absolute `revoked = 1`, which is idempotent under concurrent execution, and then raises, leaving the harness to call `ft.shrink` — the sibling that elects a leader and publishes one agreed context.

The archive's one genuine failure shows what is at stake and, awkwardly, also shows the limit of the recommendation. `real-tr-p16-full` ended with sixteen ranks in `failed_ranks`, `ok: false`, 7,200 s of wall clock, $0.0726 spent and no output: six rank roles were never bound to a worker process, and the ten that worked correctly died waiting inside a glossary `allreduce` that could never fold. Its `failures` table has zero rows and its log has zero `ft.declare_failed` events — the same empty-record end state this run reaches, reached the same way, by a collective that blocks on a timeout and records nothing about who did not arrive. Its own analysis concludes that a shrink over the ten ranks that actually had executors would have finished the job in minutes. (Ten, not twelve: twelve is the count of ranks that emitted a `coll.reduce` record, which is a different set from the set that could do work.)

But no choice among these four names would have saved it. Reading the signatures in `src/agentmpi/algorithms.py` `policy` is a parameter of `barrier` and of nothing else — `bcast`, `scatter`, `gather`, `allgather`, `reduce`, `allreduce` and `reduce_scatter` take a `timeout` and no policy at all — and the collective that killed that run was an `allreduce`. An `allreduce` behaves exactly the way this run behaves: it blocks, it times out, and it records nothing. That is the general form of what this run demonstrates. The default behaviour of every collective in AgentMPI except the central barrier is the behaviour on this page, and a harness gets the good behaviour only by placing a policy-bearing barrier ahead of the collective it actually cares about.

8 Threats to this reading

The equivalence claim is about recorded behaviour, and recorded behaviour is not all behaviour. What I can show is that these two runs produced identical event-kind counts, identical per-rank profiles, identical communication matrices, identical fabric end states and identical error strings. What I cannot show from the traces alone is that no path exists on which the two differ;

for that I am relying on the grep, which finds `BarrierPolicy.WAIT` once in `src/agentmpi`, in a condition it shares with `RAISE`. That is strong but it is a static reading, and a caller constructing the policy from a string, or third-party code branching on the enum, would not appear in it.

The round-by-round reconstruction is derived, not logged. No event records which round a message belonged to. I recovered the structure from the internal tags, which encode it as `_ampi:barrier:{generation}:{epoch}:{k}`, and checked the implied send and receive counts against the per-rank table in `generated.tex`, which matches exactly. Mapping those counts onto "rank 5 blocked waiting on rank 3" additionally assumes the $(r \pm 2^k) \bmod p$ schedule as implemented, read from the source. The arithmetic is over-determined and I am confident in it, but it is an inference and no artefact states it.

The claim that recovery was foreclosed is a source reading, not an observation. `rank_main` catches the `AmpiTimeout` and returns; it never attempts `ft.shrink` or `ft.agree`, so nothing here shows what would have happened if it had. I rely on `_agree_survivors` computing its survivor set from `get_failed` plus `ft.health`, and on the `failures` table being empty. The hole in that reading is `ft.health` itself: its lease-based detector could in principle have suspected rank 3 independently, since rank 3's lease would eventually expire. Nothing in this run exercises it, and whether it would fire inside a 15 s window depends on the lease length, which this trace does not record.

There are no agents here and nothing in this run bears on agent behaviour. `executors: {none: 8}`, zero agent calls, zero tokens, $0. The 17 messages carry one token each and share a single digest, `6b86b273ff34` — the same one-byte payload seventeen times. The failure modelled is a rank that returns instantly, which is `FAIL_STOP` and only `FAIL_STOP`, and `src/agentmpi/ft.py`'s own taxonomy is explicit that the class that matters for agent systems is `FAIL_PLAUSIBLE` — a rank that returns a confident wrong answer, which no deadline detects. These four runs exercise one row of a six-row table, and the barrier policies address that row alone. It is worth being blunt that a genuine `WAIT` implementation would not have helped with any of the other five either.

Two counters named `tokens_deferred` disagree across artefacts. `job.finish` reports 10; `metrics.json` reports 0 at the top level and surfaces the 10 as `summary.tokens_unadmitted`. Both are right under their own definitions — `analysis.py` counts tokens on rendezvous *sends*, of which there were none, while the runtime counts received tokens not admitted to a context, which is all ten receives at one token each — but the same name carries different values in two files a reader will open together. Nothing here depends on it; the same collision at a scale that matters is documented in `real-tr-p16-full`, at 50,544 against 16,082.

$n = 1$, though almost nothing in this reading is statistical. A rerun would shuffle the ordering of the 17 sends and move the wall clock by tens of milliseconds. It would not change the send and receive counts, which follow from the dissemination schedule and the victim's identity; not the empty `failures` and `collectives` tables, which follow from which branch of `algorithms.barrier` executes; not the rank named in the error, which is fixed by $(0 - 4) \bmod 8$; and not the equivalence with `mb-free__faults-raise`, which follows from the source. The 11 ms separating the two runs is the only quantity here that a second sample would meaningfully change, and it is also the only quantity that distinguishes them.